

Vulnerability Assessment and Systems Assurance Report – Preventing Injection Attacks

Preventing Injection Attacks Report

Asal Tiwari

ITIS 4221

July 22, 2026

VULNERABILITY ASSESSMENT AND SYSTEMS ASSURANCE REPORT

TABLE OF CONTENTS

<u>Section</u>	<u>Page#</u>
1.0 General Information	3
2.0 Security Enhanced API	4
3.0 Path Manipulation	5
4.0 SMTP Header Injection	7
5.0 SQL Injection	10
6.0 Command Injection	12
7.0 Log Forging	14
8.0 XPath Injection	15
9.0 Cross-Site Scripting(XSS)	17
10.0 Exception Handling	20

1.0 General Information

1.1 *Purpose*

The objective of this assessment is to identify and address injection-related vulnerabilities present in the given application. This report covers SQL Injection, Cross-Site Scripting (XSS), Command Injection, Path Manipulation, Log Forging, SMTP Header Injection, and XPath Injection. For each vulnerability, this report describes the flaw, demonstrates successful exploitation with screenshots, presents the vulnerable source code, and presents the corrected source code along with a re-run confirming the fix. The application is a Spring Boot application hosted on Apache Tomcat, running on a Parrot OS virtual machine at <http://localhost:8081/>. All attacks were performed in a controlled lab environment for educational purposes only.

2.0 Security Enhanced API

A reusable validation utility class was created to centralize input-validation logic used across multiple fixes. It exposes three static methods, each validating a string against a strict allow-list regular expression and throwing a dedicated checked exception on failure.

Implementation

```
package net.uncc.app.security;

import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.regex.Pattern;

public class SecurityEnhancedAPI {

    private static final Pattern FILENAME_PATTERN =
        Pattern.compile("^([\\w,\\s-]+\\.[A-Za-z]{3,10})$");

    private static final Pattern SAFE_STRING_PATTERN =
        Pattern.compile("^([\\p{Alnum}\\p{Space}]{0,1024})$");

    private static final Pattern EMAIL_PATTERN =
        Pattern.compile("[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}");

    public static String GetFilename(String input) throws WrongFileName {
        if (input != null && FILENAME_PATTERN.matcher(input).matches()) {
            return input;
        }
        throw new WrongFileName("Invalid filename: " + encodeForLog(input));
    }

    public static String GetSafeString(String input) throws WrongSafeString {
        if (input != null && SAFE_STRING_PATTERN.matcher(input).matches()) {
            return input;
        }
        throw new WrongSafeString("Invalid safe string: " + encodeForLog(input));
    }

    public static String GetEmail(String input) throws WrongEmail {
        if (input != null && EMAIL_PATTERN.matcher(input).matches()) {
            return input;
        }
        throw new WrongEmail("Invalid email: " + encodeForLog(input));
    }

    private static String encodeForLog(String input) {
        if (input == null) {
```

```

        return "null";
    }
    return URLEncoder.encode(input, StandardCharsets.UTF_8);
}
}

```

Custom Exceptions

```

public class WrongFileName extends Exception {
    public WrongFileName(String message) { super(message); }
}

public class WrongEmail extends Exception {
    public WrongEmail(String message) { super(message); }
}

public class WrongSafeString extends Exception {
    public WrongSafeString(String message) { super(message); }
}

```

3.0 Path Manipulation

The file-viewing endpoint concatenated user-supplied input directly into a file path with no validation, allowing directory-traversal sequences (“../”) to escape the intended files/directory and read arbitrary files on the server, including the application's own configuration file.

The filename “./application.properties” was submitted through the Path Manipulation form. The application's database configuration, including the datasource URL and username, was returned in the response.

Path Manipulation

Home
Operation

Access files and directories

Access database information to exploit: ../application.properties

Inject Command

```
## Spring view resolver set up server.port=8081 spring.mvc.view.prefix=/WEB-INF/jsp/ spring.mvc.view.suffix=.jsp ##
Spring DATASOURCE (DataSourceAutoConfiguration & DataSourceProperties) spring.datasource.url = jdbc:mysql://localhost:3306/vulnerability?useSSL=false spring.datasource.username = root spring.datasource.password =
spring.jpa.show-sql=true ## Hibernate Properties # The SQL dialect makes Hibernate generate better SQL for the
chosen database spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.MySQL5InnoDBDialect # Hibernate ddl
auto (create, create-drop, validate, update) spring.jpa.hibernate.ddl-auto = update
```

Filename:

Submit

Vulnerable Code

```

@GetMapping("/viewFile")
@ResponseBody
public Map<String, String> view_file(@RequestParam String file_name) throws Exception {
    Map<String, String> response_data = new HashMap<String, String>();

    try {
        Resource resource = resourceLoader.getResource("classpath:files/" + file_name); // VULNERABLE
        File file = resource.getFile();
        String text = new String(Files.readAllBytes(file.toPath()));

        response_data.put("status", "success");
        response_data.put("msg", text);
        return response_data;
    } catch (IOException e) {
        e.printStackTrace();
        response_data.put("status", "error");
        response_data.put("msg", "No output found");
        return response_data;
    }
}

```

The filename parameter is concatenated directly into the file path with no check for “../” or any other disallowed character, so any path the underlying OS can resolve is reachable.

Fixed Code

```

@GetMapping("/viewFile")
@ResponseBody
public Map<String, String> view_file(@RequestParam String file_name) throws Exception {
    Map<String, String> response_data = new HashMap<String, String>();

    try {
        String safe_file_name = SecurityEnhancedAPI.GetFilename(file_name);

        Resource resource = resourceLoader.getResource("classpath:files/" + safe_file_name);
        File file = resource.getFile();
        String text = new String(Files.readAllBytes(file.toPath()));

        response_data.put("status", "success");
        response_data.put("msg", text);
        return response_data;
    } catch (WrongFileName e) {
        String encoded_input = URLEncoder.encode(file_name, StandardCharsets.UTF_8);
        logger.warn("Rejected invalid filename input: {}", encoded_input);
        response_data.put("status", "error");
        response_data.put("msg", "Invalid filename");
        return response_data;
    } catch (IOException e) {
        e.printStackTrace();
        response_data.put("status", "error");
    }
}

```

```

        response_data.put("msg", "No output found");
        return response_data;
    }
}

```

The GetFilename() method validates input against the pattern `^[\w,\s-]+\.[A-Za-z]{3,10}$`, which permits word characters, spaces, commas and hyphens followed by a short alphabetic extension - path-traversal sequences such as “../” cannot match this pattern.

The identical payload was resubmitted after the fix was applied. The request was rejected and no file contents were returned.

The screenshot shows a web application with a dark theme. At the top, there is a navigation bar with 'Path Manipulation' as the active tab, and 'Home' and 'Operation' as other options. Below the navigation bar, the main heading is 'Access files and directories'. Underneath this heading, a message reads: 'Access database information to exploit: ../application.properties'. Further down, the heading 'Inject Command' is displayed, followed by the text 'Invalid filename'. Below this, there is a form with a label 'Filename:' and a text input field containing the value '../application.properties'. At the bottom of the form is a green 'Submit' button.

4.0 SMTP Header Injection

The comment-form endpoint built simulated email headers by concatenating unvalidated user input. Because the email field was inserted directly next to a literal newline delimiter, an attacker could embed additional newline sequences to inject an arbitrary extra header line.

The value `attacker@test.com\nBcc:victim@test.com` was submitted in the Email field. The rendered output contained an extra, fully attacker-controlled “Bcc:” header line that was never part of the legitimate form.

SMTP Header Injection

Home

From:hiii
 to:attacker@test.com
 Bcc:victim@test.com
 Message:

First Name:

Email:

Comment:

Submit

Vulnerable Code

```

@GetMapping("/form")
@ResponseBody
public String smtp_header_submit(@RequestParam String customer_firstName,@RequestParam String
customer_email,@RequestParam String customer_comments) {
    String name = customer_firstName;
    String email = customer_email;
    String comment = customer_comments;
    String to = "root@localhost";
    String subject = "My Subject";

    String headers = "From:" + name + "\n" + " to:" + email + "\n"; // VULNERABLE
    String[] split = headers.split("\\n");
    String y="";
    for (int i = 0; i < split.length; i++) {
        y += split[i];
        y += "<br>";
    }
    System.out.println(y);
    return y + " Message:" + comment;
}

```

The email value is placed directly next to the newline delimiter with no check for additional embedded newline or colon characters, allowing extra header lines to be injected.

Fixed Code

```

@GetMapping("/form")
@ResponseBody
public String smtp_header_submit(@RequestParam String customer_firstName,
                                @RequestParam String customer_email,
                                @RequestParam String customer_comments) {

```

```

String name = customer_firstName;
String comment = customer_comments;
String to = "root@localhost";
String subject = "My Subject";

try {
    String safe_email = SecurityEnhancedAPI.GetEmail(customer_email);

    String headers = "From:" + name + "\\n" + " to:" + safe_email + "\\n";
    String[] split = headers.split("\\n");
    String y = "";
    for (int i = 0; i < split.length; i++) {
        y += split[i];
        y += "<br>";
    }
    System.out.println(y);
    return y + " Message:" + comment;
} catch (WrongEmail e) {
    String encoded_input = URLEncoder.encode(customer_email, StandardCharsets.UTF_8);
    logger.warn("Rejected invalid email input: {}", encoded_input);
    return "Invalid email address";
}
}

```

GetEmail() validates the input against a standard email address pattern that has no room for a newline or colon character, so any header-injection payload fails validation immediately and never reaches the header-building logic.

The identical payload was resubmitted after the fix was applied. Validation rejected it before any header string was built.

SMTP Header Injection

Home

Invalid email address

First Name:

Email:

Comment:

Submit

5.0 SQL Injection

The employee login and address-update endpoints-built SQL statements by concatenating user input directly into the query string. A helper method also accepted a parameters argument but never actually applied it, giving a false impression of parameterization.

The username `abr04' --` was submitted with an empty password. The trailing SQL comment (`--`) stripped the password check entirely, authenticating as the employee "Abraham" without any valid credentials. While logged in as Abraham, the address `547 DEF Drive, CLT, NC, 28262' where username = 'bob03' #` was submitted to the Update Address form. The injected WHERE clause overrode the intended condition, updating a different employee's (bob03) address instead of the logged-in user's own.

SQL Injection
Home
Login as Employee
Update Information

Username	Password	First Name	Last Name	Department	Address
abr04	123	Abraham	Abraham	Development	647 DEF Drive, CLT, NC, 28262
bob03	456	Bob	Franco	Marketing	646 DEF Drive, CLT, NC, 28262
john05	789	John	Smith	Marketing	648 DEF Drive, CLT, NC, 28262
pau01	101	Paulina	Travers	Accounting	644 ABC Drive, CLT, NC, 28262
tob02	112	Tobi	Barnett	Development	645 DEF Drive, CLT, NC, 28262

"Successfully logged in as Abraham Abraham"

Username: Password:

Log In Reset

SQL Injection
Home
Login as Employee
Update Information

Update Address

Username	First Name	Last Name	Department	Address
abr04	Abraham	Abraham	Development	647 DEF Drive, CLT, NC, 28262
bob03	Bob	Franco	Marketing	547 DEF Drive, CLT, NC, 28262
john05	John	Smith	Marketing	648 DEF Drive, CLT, NC, 28262
pau01	Paulina	Travers	Accounting	644 ABC Drive, CLT, NC, 28262
tob02	Tobi	Barnett	Development	645 DEF Drive, CLT, NC, 28262

Address:

Submit

Vulnerable Code

```
private Object findDataFromDatabase(String queryString, Object param) {
    try {
        return jdbcTemplate.queryForList(queryString); // param accepted but never used
    } catch (Exception e) {
        e.printStackTrace();
        return new ArrayList<>();
    }
}

@PostMapping("/")
@ResponseBody
public Map<String, String> sql_logged_in(@RequestParam String employee_username, @RequestParam String employee_password, HttpServletRequest request) {
    String queryString = "SELECT * From Employees where username = '" + employee_username + "' and password = '" + employee_password + "'"; // VULNERABLE
    Object[] parameters = {employee_username, employee_password};
    List<Map> listOfemployee = (List<Map>) findDataFromDatabase(queryString, parameters);
    ...
}

@PostMapping("/update_address")
@ResponseBody
```

```

public String update_address(@RequestParam String updated_address, HttpServletRequest request) throws
Exception {
    ...
    String updateQuery = "UPDATE Employees SET address = '' + updated_address + '' WHERE
username = '' + loggedInUser + ''"; // VULNERABLE
    updatedEmpInfo = jdbcTemplate.update(updateQuery);
    ...
}

```

Both the login query and the address-update query are built by string concatenation, so a single quote in the input can close the intended string literal and let the attacker supply their own SQL logic, such as a comment that removes the password check.

Fixed Code

```

private Object findDataFromDatabase(String queryString, Object... params) {
    try {
        return jdbcTemplate.queryForList(queryString, params);
    } catch (Exception e) {
        e.printStackTrace();
        return new ArrayList<>();
    }
}

@PostMapping("/")
@ResponseBody
public Map<String, String> sql_logged_in(@RequestParam String employee_username, @RequestParam
String employee_password, HttpServletRequest request) {
    String queryString = "SELECT * From Employees where username = ? and password = ?";
    List<Map> listOfemployee = (List<Map>) findDataFromDatabase(queryString, employee_username,
employee_password);
    ...
}

@PostMapping("/update_address")
@ResponseBody
public String update_address(@RequestParam String updated_address, HttpServletRequest request) throws
Exception {
    ...
    String updateQuery = "UPDATE Employees SET address = ? WHERE username = ?";
    updatedEmpInfo = jdbcTemplate.update(updateQuery, updated_address, loggedInUser);
    ...
}

```

Both statements now use ? placeholders with values bound separately via JdbcTemplate, so user input is always treated as literal data, never as SQL syntax.

The identical payload was resubmitted after the fix was applied.

SQL Injection

[Home](#)
[Login as Employee](#)
[Update Information](#)

Username	Password	First Name	Last Name	Department	Address
abr04	123	Abraham	Abraham	Development	647 DEF Drive, CLT, NC, 28262
bob03	456	Bob	Franco	Marketing	646 DEF Drive, CLT, NC, 28262
joh05	789	John	Smith	Marketing	648 DEF Drive, CLT, NC, 28262
pau01	101	Paulina	Travers	Accounting	644 ABC Drive, CLT, NC, 28262
tob02	112	Tobi	Barnett	Development	645 DEF Drive, CLT, NC, 28262

"No such customer found"

Username: Password:

[Log In](#)
[Reset](#)

SQL Injection

[Home](#)
[Login as Employee](#)
[Update Information](#)

6.0 Command Injection

The network-diagnostics endpoint built a shell command string by concatenating user input and executed it through `/bin/bash -c`, allowing shell metacharacters such as `&&` to chain in arbitrary additional operating-system commands.

The value `"8.8.8.8 && ls && whoami"` was submitted as the IP address. All three commands executed: the ping output, a full directory listing of the application's working directory, and the current operating-system username.

Command Injection

[Home](#)
[Operation](#)

```

3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 22.313/23.552/24.228/0.877 ms
Penetration_test.iml
README.md
hs_err_pid26611.log
hs_err_pid26637.log
hs_err_pid27136.log
hs_err_pid27147.log
hs_err_pid2758.log
hs_err_pid2832.log
logs
pom.xml
src
target
user
  
```

IP Address:

[Submit](#)

Vulnerable Code

```

@PostMapping("/output/")
@ResponseBody
public Object command_injected(@RequestParam String ip_address) {
  
```

```

Map<String, String> response_data = new HashMap<String, String>();
try {
    String output = "";
    String[] command = {"/bin/bash", "-c", "ping -c 3 " + ip_address}; // VULNERABLE
    Process proc = Runtime.getRuntime().exec(command);
    proc.waitFor();
    ...
} catch (Exception e) { ... }
}

```

The IP address is concatenated into a single string and handed to a shell (/bin/bash -c), so shell operators such as && in the input are interpreted as command separators rather than as part of a hostname.

Fixed Code

```

@PostMapping("/output/")
@ResponseBody
public Object command_injected(@RequestParam String ip_address) {
    Map<String, String> response_data = new HashMap<String, String>();
    try {
        String output = "";

        ProcessBuilder processBuilder = new ProcessBuilder();
        processBuilder.command("ping", "-c", "3", ip_address);
        Process proc = processBuilder.start();

        proc.waitFor();
        ...
    } catch (Exception e) { ... }
}

```

Because ip_address is passed as one literal argument directly to the ping binary with no shell involved, metacharacters such as && have no special meaning; the entire string is treated as a single (invalid) hostname.

The identical payload was resubmitted after the fix was applied. Ping failed on the whole string as a single invalid hostname; no additional commands executed.

Command Injection

Home Operation

To attack use chaining command: &, &&, |, ||

(example: 8.8.8.8 && ls && whoami)

Accepted commands:

- **whoami**: displays the username of the current user.
- **ifconfig**: displays current network configuration information.
- **ping -c 4 8.8.8.8**: acts as a test to see if a networked device is reachable.
- **ls**: lists directory contents by names.

Inject Command

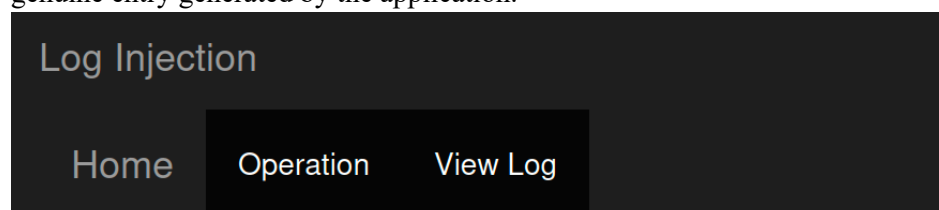
ping: 8.8.8.8 && ls && whoami: Name or service not known

IP Address:

7.0 Log Forging

When a submitted value failed integer parsing, the exception handler logged the raw, previously URL-decoded input directly. Because the value had already been decoded, an attacker could submit an encoded newline (%0a) that became a literal newline in the log file, forging what appears to be a separate, legitimate log entry.

The payload `twenty-one%0a%0aINFO:+User+logged+out%3dbadguy` was submitted. Viewing the log afterward showed a fabricated "INFO: User logged out=badguy" line that is indistinguishable from a genuine entry generated by the application.



Logged data:

INFO - After exception: twenty-one

INFO: User logged out=badguy

Vulnerable Code

```
@PostMapping("/")
@ResponseBody
public Object log_injected(@RequestParam String log_value) {
    ...
    try {
        ...
        log_value = java.net.URLDecoder.decode(log_value, StandardCharsets.UTF_8.name());
        Integer parsed_log_value = Integer.parseInt(log_value);
        logger.info("Value to log: " + parsed_log_value);
        ...
    } catch (Exception e) {
        logger.info("After exception: " + log_value); // VULNERABLE
        ...
    }
}
```

The value logged in the catch block is the raw, already-decoded input, so any newline characters it contains become real line breaks in the log file.

Fixed Code

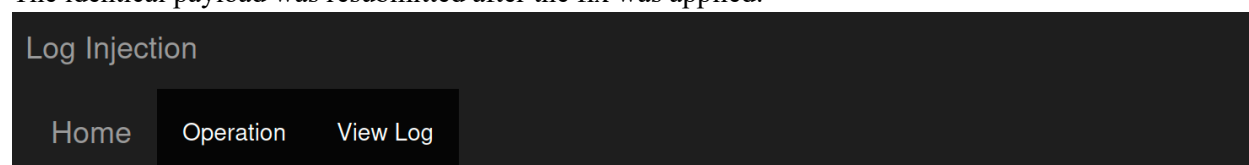
```

    } catch (Exception e) {
        String safe_log_value = URLEncoder.encode(log_value, StandardCharsets.UTF_8);
        logger.info("After exception: " + safe_log_value);
        ...
    }

```

URLEncoder.encode() converts newline and other special characters into harmless percent-encoded text before the value is written to the log, so the entire payload regardless of its contents, can only ever occupy a single log line and can never forge a new entry.

The identical payload was resubmitted after the fix was applied.

**Logged data:**

INFO - After exception: twenty-one

INFO: User logged out=badguy

INFO - After exception: twenty-one%0A%0AINFO%3A+User+logged+out%3Dbadguy

8.0 XPath Injection

The customer lookup endpoint built an XPath expression by concatenating user input directly into the query string, allowing XPath boolean logic to be injected and bypass the intended email filter entirely.

The payload `mpurba@xyz.com' or 1 = '1` was submitted as the email. The injected always-true condition matched every customer node in the XML data store, returning all four customer IDs from a single lookup instead of the one that was intended.

XPath Injection

Home
Operation

```

<email>basel@xyz.com</email>
<id>886462</id>
<password>912</password>
<name>Basel Abdeen</name>
<phone>980-846-2468</phone>
<amount>60000</amount>
</customer>
</customers>

```

Inject XPath

(Example 1: `mpurba@xyz.com` or `email = 'ashu@xyz.com'`)

(Example 2: `mpurba@xyz.com` or `1 = '1'`)

[886459, 886460, 886461, 886462]

Email:

Submit

Vulnerable Code

```

XPathExpression expression = xpath.compile("/customers/customer[email = '" + email_address +
    "']/id/text()"); // VULNERABLE
NodeList nodes = (NodeList) expression.evaluate(doc, XPathConstants.NODESET);

```

The email value is concatenated directly into the XPath expression string, so a single quote in the input can close the intended literal and let the attacker inject their own XPath boolean logic.

Fixed Code

```

SimpleVariableResolver resolver = new SimpleVariableResolver();
resolver.addVariable(new QName(null, "email_val"), email_address);
xpath.setXPathVariableResolver(resolver);

List<String> id_list = new ArrayList<>();
XPathExpression expression = xpath.compile("/customers/customer[email = $email_val]/id/text()");
NodeList nodes = (NodeList) expression.evaluate(doc, XPathConstants.NODESET);
for (int i = 0; i < nodes.getLength(); i++)
    id_list.add(nodes.item(i).getNodeValue());

```

The submitted email address is now bound to a query variable (\$email_val) using a SimpleVariableResolver, instead of being concatenated directly into the XPath expression string. The XPath engine only ever treats the submitted value as a literal string to compare against, so characters like quotes and boolean operators in the input no longer affect how the query itself is parsed. The identical payload was resubmitted after the fix was applied. The expression returned an empty result set.

XPath Injection

Home
Operation

```

<email>basel@xyz.com</email>
<id>886462</id>
<password>912</password>
<name>Basel Abdeen</name>
<phone>980-846-2468</phone>
<amount>60000</amount>
</customer>
</customers>

```

Inject XPath

(Example 1: `mpurba@xyz.com` or `email = 'ashu@xyz.com'`)

(Example 2: `mpurba@xyz.com` or `1 = '1'`)

[]

Email:

9.0 Cross-Site Scripting(XSS)

Three endpoints reflected user input directly back into the response with no output encoding, each rendering into a different context: the HTML page body, an HTML < textarea>, and a JavaScript event-handler attribute. Because each context requires a different encoding strategy, all three were tested independently.

The payload `<script>alert(1)</script>` was submitted in the "By body" form. The script executed and produced a JavaScript alert dialog.

Cross-Site Scripting

Home
XSS Operation

Allowance of html/script tags as input without Sanitization

To avoid this attack, encode/sanitize your input before sending back to Front-end to display.

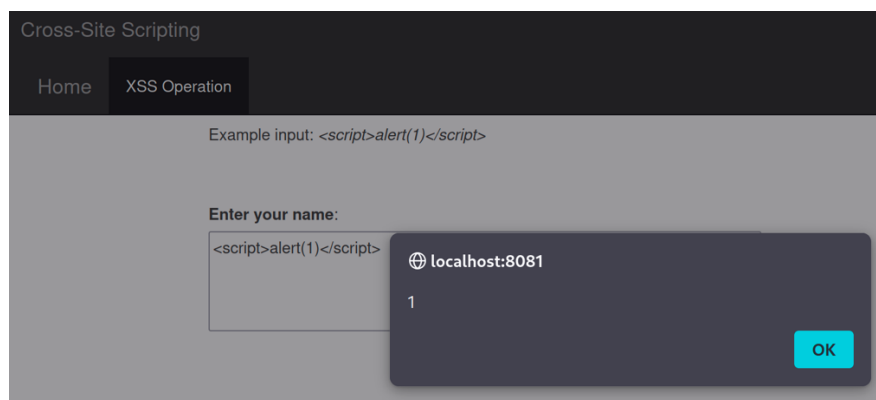
By body

Example input: `<script>alert(1)</script>`

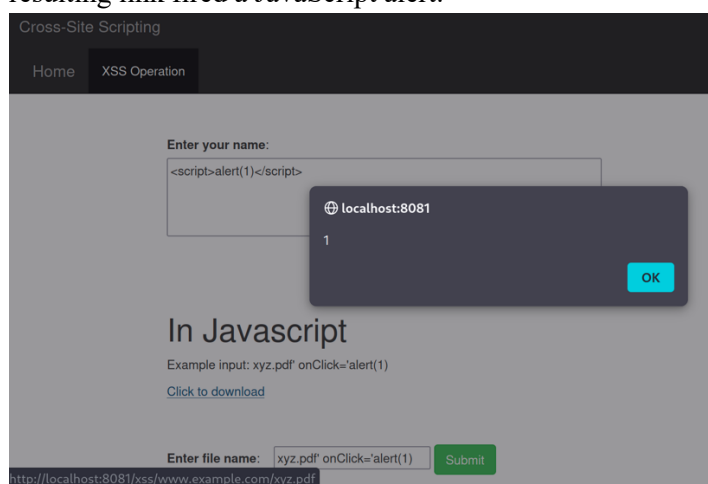
Enter your name:

localhost:8081
1
OK

The same payload was submitted in the "Into Textarea" form. The raw, un-encoded script tag was reflected directly into the page.



The payload `xyz.pdf' onClick='alert(1)` was submitted as a file name. The single quote closed the intended attribute value early, injecting a live `onClick` handler into the generated download link. Clicking the resulting link fired a JavaScript alert.



Vulnerable Code

```
@GetMapping("/body_xss")
@ResponseBody
public String body_xss(@RequestParam String body_tagVal) throws Exception {
    return body_tagVal; // VULNERABLE (reflected into HTML body context)
}

@GetMapping("/textarea_xss")
@ResponseBody
public Object textarea_xss(@RequestParam String textarea_tagVal) throws Exception {
    return textarea_tagVal; // VULNERABLE (reflected into HTML textarea context)
}

@GetMapping("/js_xss")
@ResponseBody
public Object js_xss(@RequestParam String js_tagVal) throws Exception {
    return js_tagVal; // VULNERABLE (reflected into JavaScript context)
}
```

All three methods return the submitted value with no encoding applied at all, so any HTML or script content the user submits is reflected back and rendered exactly as written, regardless of which context it lands in.

Fixed Code

```
@GetMapping("/body_xss")
@ResponseBody
public String body_xss(@RequestParam String body_tagVal) throws Exception {
    return escapeHtml(body_tagVal);
}

@GetMapping("/textarea_xss")
@ResponseBody
public Object textarea_xss(@RequestParam String textarea_tagVal) throws Exception {
    return escapeHtml(textarea_tagVal);
}

@GetMapping("/js_xss")
@ResponseBody
public Object js_xss(@RequestParam String js_tagVal) throws Exception {
    return escapeJavaScript(js_tagVal);
}
```

The body and textarea endpoints use `escapeHtml()`, which converts characters such as `<` and `>` into their HTML entity equivalents so a submitted `<script>` tag is displayed as inert text rather than executed. The JavaScript endpoint instead uses `escapeJavaScript()`, which escapes quotes and backslashes so a submitted quote character can no longer close the surrounding attribute early.

The identical body and textarea payloads were resubmitted after the fix was applied; both now display as harmless, encoded text with no alert firing.

Cross-Site Scripting

Home XSS Operation

Allowance of html/script tags as input without Encoding/Sanitization
To avoid this attack, encode/sanitize your input before sending back to Front-end to display.

By body
Example input: `<script>alert(1)</script>`
`<script>alert(1)</script>`

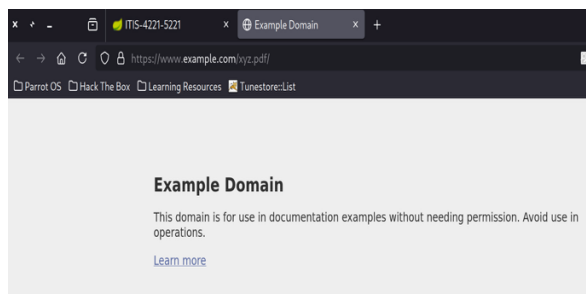
Enter your name:

Into Textarea
Example input: `<script>alert(1)</script>`
`<script>alert(1)</script>`

Enter your name:

In Javascript
Example input: `xyz.pdf onClick=alert(1)`
[Click to download](#)

Enter file name:

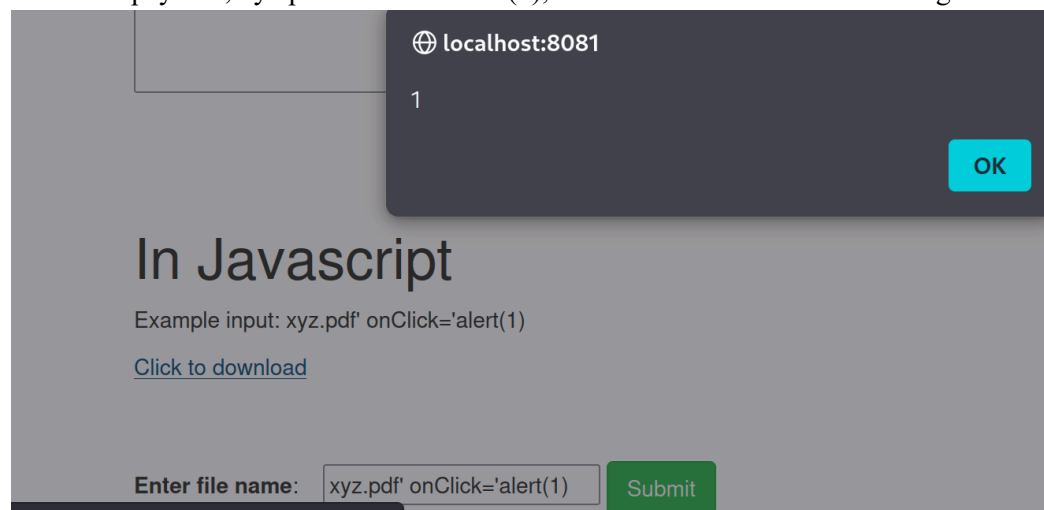


Demonstration: Incorrect Encoding Does Not Mitigate XSS

To demonstrate that the encoding function must match its output context, the JavaScript-context endpoint was temporarily changed to use `escapeHtml()`, the encoder that correctly protects the HTML body and textarea contexts above, in place of the correct `escapeJavaScript()`.

```
@GetMapping("/js_xss")
@ResponseBody
public Object js_xss(@RequestParam String js_tagVal) throws Exception {
    return escapeHtml(js_tagVal); // incorrect encoder for this context
}
```

The same payload, `xyz.pdf' onClick='alert(1)`, was resubmitted and the resulting link clicked.



The alert still fired. HTML entity encoding neutralizes characters that are dangerous inside HTML markup, such as `<` and `>`, but it does not neutralize the single-quote character in a way that prevents it from closing a single-quoted JavaScript attribute value. This confirms that encoding alone is not sufficient, the encoding function must be chosen to match where the data is rendered. Only `escapeJavaScript()` correctly blocks this attack, as shown above. The endpoint was reverted to `escapeJavaScript()` before final testing.

10.0 Exception Handling

Exceptions raised by `SecurityEnhancedAPI` are handled consistently everywhere the class is used: the offending input is logged in URL-encoded form, and the request is rejected with a generic error message that reveals no internal details back to the client.

Path Manipulation

```
} catch (WrongFileName e) {
    String encoded_input = URLEncoder.encode(file_name, StandardCharsets.UTF_8);
    logger.warn("Rejected invalid filename input: {}", encoded_input);
    response_data.put("status", "error");
    response_data.put("msg", "Invalid filename");
    return response_data;
}
```

SMTP Header Injection

```
} catch (WrongEmail e) {  
    String encoded_input = URLEncoder.encode(customer_email, StandardCharsets.UTF_8);  
    logger.warn("Rejected invalid email input: {}", encoded_input);  
    return "Invalid email address";  
}
```

In both cases the raw, offending input is never echoed back to the client, and the copy written to the application log is URL-encoded so that it cannot itself be used to forge additional log entries, as described in Section 7.0.